Deep Guide to Notebooks 00–04

Hateful Meme Detection Project

From prerequisites to OCR pipeline

Project:	hateful-meme-project
Scope:	notebooks 00_setup through 04_ocr_pipeline
Focus:	prerequisites, rationale, code walkthrough, interpretation
Prepared:	April 7, 2026

This document is written for a reader starting from scratch. It explains the conceptual background, the Python and machine learning prerequisites, the dataset conventions, and then the purpose and internal logic of notebooks 00, 01, 02, 03, and 04 in detail.

Contents

1 How to Read This Project From Scratch **2**

1.1 What Problem Is This Project Solving? 2

1.2 Why Not Just Use Text-Only or Image-Only Models? 3

2 Prerequisites **4**

2.1 Python and Notebook Basics 4

2.2 Machine Learning Prerequisites 4

2.3 Deep Learning Prerequisites 5

2.4 Dataset Prerequisites 6

2.5 Why OCR Matters at All 6

3 Notebook 00: Environment Setup and Project Verification **7**

3.1 High-Level Goal 7

3.2 Cell Group 1: Dataset Path Detection 7

3.3 Cell Group 2: Reproducibility Seeding 8

3.4 Cell Group 3: Project Root Detection 8

3.5 Cell Group 4: Device Check 9

3.6 Cell Group 5: Package Verification 9

3.7 Cell Group 6: Dataset File Verification 9

3.8 Cell Group 7: Load One Sample 10

3.9 Cell Group 8: Source Module Import Check 10

3.10 Cell Group 9: Config and Checkpoint Verification 10

3.11 What Notebook 00 Teaches 10

4 Notebook 01: Problem Scope and Design Decisions **12**

4.1 Task Definition 12

4.2 Scope Boundary 12

4.3 Dataset Description . . . 13
4.4 Success Metrics . . . 13
4.5 Architecture Decisions . . . 13
4.6 Imbalance Handling . . . 14
4.7 Responsible AI Layer . . . 14
4.8 Deliverables Framing . . . 14
4.9 Final Sanity Check Cell . . . 14

5 Notebook 02: Data Inspection and Verification 15

5.1 Why Data Inspection Comes Before EDA . . . 15
5.2 Loading the Splits . . . 15
5.3 Image Path Resolution . . . 15
5.4 Class Balance Check . . . 16
5.5 Image Existence and Corruption Check . . . 16
5.6 Duplicate and Leakage Checks . . . 16
5.7 Missing or Empty Text Fields . . . 17
5.8 Random Sample Inspection . . . 17
5.9 Image Resolution Analysis . . . 17
5.10 Modality-Need Analysis . . . 17
5.11 What Notebook 02 Teaches . . . 18

6 Notebook 03: Exploratory Data Analysis 19

6.1 Why EDA Matters Here . . . 19
6.2 Class Distribution . . . 19
6.3 Text Length Distributions . . . 20
6.4 Detailed Text Statistics . . . 20
6.5 Violin Plots and Statistical Testing . . . 20
6.6 Vocabulary Richness . . . 20
6.7 Zipf-Law Visualization . . . 21
6.8 Bigram and Trigram Analysis . . . 21
6.9 Surface Characteristics . . . 21
6.10 Text Duplication and Overlap . . . 21
6.11 Word Frequencies and Word Clouds . . . 22
6.12 Demographic Mention Analysis . . . 22

6.13 Confounder Check . . . 22
6.14 What Notebook 03 Teaches . . . 23

7 Notebook 04: OCR Pipeline 24

7.1 The Core Tension . . . 24
7.2 Step 1: Audit Existing Text Quality . . . 24
7.3 Step 2: Inspect Suspiciously Short Entries . . . 25
7.4 Step 3: Install and Initialize EasyOCR . . . 25
7.5 Step 4: Compare Dataset Text With EasyOCR on a Sample . . . 25
7.6 Step 5: OCR Quality Heuristic . . . 25
7.7 Step 6: Final Decision Logic . . . 26
7.8 Step 7: Optional Full OCR Pass . . . 26
7.9 Step 8: Visual Overlay Output . . . 26
7.10 What Notebook 04 Teaches . . . 26

8 Cross-Notebook Patterns and Design Lessons 27

8.1 Each Notebook Is Self-Sufficient . . . 27
8.2 Environment Robustness Is Treated as Part of the Project . . . 27
8.3 Analysis Precedes Modeling . . . 27
8.4 The Project Is Already Framed as More Than a Benchmark . . . 28

9 Concise Notebook-by-Notebook Summary 29

10 Final Interpretation 30

A Key Numbers Used in This Report 31

B Suggested Reading Order for a Beginner 32

Chapter 1

How to Read This Project From Scratch

This project studies hateful meme detection as a **multimodal classification** problem. A meme is not just an image and not just text. Meaning often arises from the interaction between the visual content and the overlaid caption. Because of that, the project is structured as a pipeline of notebooks that move from environment validation to problem definition, then data inspection, deeper exploratory analysis, and finally OCR quality analysis.

The five notebooks covered here are:

1. `00_setup.ipynb`: environment and data sanity checking.
2. `01_problem-scope.ipynb`: task definition and design decisions.
3. `02_data-inspection.ipynb`: raw dataset loading and validation.
4. `03_eda.ipynb`: exploratory data analysis of language and image-related metadata.
5. `04_ocr_pipeline.ipynb`: audit of text extraction quality and OCR fallback strategy.

There is one subtle but important detail: the overview table inside notebook 00 appears to come from an older naming scheme. The current workspace contains notebooks named `01_problem-scope`, `02_data-inspection`, `03_eda`, and `04_ocr_pipeline`. That mismatch is not a conceptual problem, but it is useful to know when reading the notebooks.

1.1 What Problem Is This Project Solving?

The target task is binary classification:

$$f(x_{image}, x_{text}) \rightarrow y \in \{0, 1\}$$

where:

- x_{image} is the meme image,

- x_{text} is the associated meme text,
- $y = 1$ means hateful,
- $y = 0$ means non-hateful.

This looks simple on paper, but it is difficult in practice because hateful meaning is frequently **contextual**. An image alone may seem harmless. The text alone may also seem mild. Yet when combined, the pair becomes hateful. A good model therefore needs to learn not only image features and text features, but also cross-modal interactions.

1.2 Why Not Just Use Text-Only or Image-Only Models?

If the problem were only text classification, then a standard NLP pipeline such as TF-IDF with logistic regression or a transformer encoder might be enough. If it were only image classification, a CNN or vision transformer could suffice. Hateful memes are harder because the label depends on joint interpretation.

Suppose one meme shows a neutral image and a hateful caption. Another shows a visually offensive stereotype but a benign or sarcastic caption that changes the meaning. The project explicitly treats these cases as evidence that a simple unimodal system is not enough. That is why later notebooks move toward CLIP-based multimodal fusion.

Chapter 2

Prerequisites

This chapter explains the minimum background a student needs before the notebooks will make full sense.

2.1 Python and Notebook Basics

The notebooks are written in Python and assume comfort with:

- importing modules,
- working with paths via `pathlib.Path`,
- reading JSON Lines files,
- using `pandas` DataFrames,
- plotting with `matplotlib`,
- understanding the difference between CPU and GPU execution.

If any of those are unfamiliar, here is the minimum practical understanding:

- A `.ipynb` notebook is an interactive document with **code cells** and **markdown cells**.
- `Path` objects make file-system logic safer than manual string concatenation.
- A JSONL file stores one JSON object per line. It is common for machine learning datasets because it streams well and is easy to parse.
- A DataFrame is a table-like structure where each row is one sample and each column is one field.

2.2 Machine Learning Prerequisites

These notebooks do not start training yet, but they already assume familiarity with a few ML ideas:

- **Train/dev/test split**: training data fits the model, dev data tunes decisions, test data estimates final generalization.
- **Class imbalance**: when one label appears more often than the other, naive accuracy can be misleading.
- **Feature extraction**: encoders turn raw images and raw text into vectors.
- **Metrics**: AUROC, F1, and accuracy capture different aspects of model behavior.

For binary classification, the model outputs either a score or probability for the positive class. A threshold then maps that score to class 0 or 1. AUROC is useful because it measures ranking quality across all thresholds, not just one.

2.3 Deep Learning Prerequisites

The requirements file lists the main libraries:

- `torch`, `torchvision`, `torchaudio`
- `transformers`, `accelerate`, `sentencepiece`
- `numpy`, `pandas`, `scikit-learn`, `tqdm`
- `Pillow`, `matplotlib`, `seaborn`, `pyyaml`
- `ipykernel`, `jupyter`

However, the notebooks themselves also use a few packages that are not emphasized in `requirements.txt`:

- `wordcloud` in notebook 03,
- `gradio` in notebook 00 package checks,
- `ipywidgets` in notebook 00 package checks,
- `scipy` for a Mann-Whitney test in notebook 03,
- `easyocr` in notebook 04.

So if a reader wants the notebooks to run completely from scratch, the operational prerequisite is broader than the base requirements file suggests.

Practical prerequisite: the setup notebook can install some missing packages dynamically, but a reproducible project usually benefits from pinning all notebook dependencies in one place. That matters especially for `easyocr`, `wordcloud`, and `scipy`.

2.4 Dataset Prerequisites

The project expects the Meta Hateful Memes dataset in a folder structure that contains:

- `train.jsonl`
- `dev.jsonl`
- `test.jsonl`
- an image directory named `img/` or `images/`

Each JSONL line is a meme sample with fields such as:

```
{
  "id": 12345,
  "img": "img/12345.png",
  "label": 1,
  "text": "some meme text"
}
```

This means the project treats the dataset not as an opaque blob, but as a structured collection where image references and text annotations must agree.

2.5 Why OCR Matters at All

Many meme datasets need OCR because the image itself contains the text. In this project, the dataset already ships with a `text` field that is described as human-cleaned OCR. Notebook 04 still checks OCR quality because:

- the text field could be incomplete,
- OCR errors can distort label-relevant cues,
- downstream preprocessing depends heavily on text quality.

So OCR in this project is not the primary source of text by default. It is an audit and fallback mechanism.

Chapter 3

Notebook 00: Environment Setup and Project Verification

Notebook 00 is the operational gatekeeper for the rest of the project. It is meant to be run before other notebooks and answers a simple question: **is the environment sane enough to trust later results?**

3.1 High-Level Goal

The notebook verifies:

- the project root,
- the Python interpreter and platform,
- GPU availability,
- required packages,
- presence of dataset files,
- existence of image files,
- the ability to import source modules,
- optional config and checkpoint files.

This may look mundane, but it is one of the most important engineering steps in the pipeline. Many machine learning notebooks fail not because the model is wrong, but because the runtime path, package versions, or dataset mount is wrong.

3.2 Cell Group 1: Dataset Path Detection

The first major code cell defines a robust data-directory discovery routine. It checks whether the code is running on Kaggle, supports explicit environment variable overrides, and searches recursively under `/kaggle/input` when necessary.

The logic is effectively:

$$\text{DATA_DIR} = \text{best available folder containing train JSONL and an image directory}$$

This is good engineering for notebook portability. The same notebook may run:

- on a local Windows machine,
- in a Kaggle notebook,
- from a repo root,
- or from inside a notebook subfolder.

Instead of hard-coding one path, the notebook infers the dataset root by checking simple invariants: a dataset folder must contain images and at least the train split.

3.3 Cell Group 2: Reproducibility Seeding

The next cell sets seeds for Python's `random`, NumPy, and PyTorch. This addresses reproducibility. Without this step, random sampling, shuffling, or model initialization can vary between runs.

Conceptually, the notebook tries to reduce variance in any process that depends on pseudo-random number generation:

$$\text{seed} = 42 \Rightarrow \text{repeatable sampling and initialization behavior}$$

This does not guarantee bit-for-bit reproducibility in every GPU context, but it is the correct baseline practice.

3.4 Cell Group 3: Project Root Detection

The notebook then tries to detect the repository root by checking whether `src/`, `configs/`, or `config/` directories exist. This matters because notebook execution often begins in a subdirectory rather than at the repository root.

The code inserts the root into `sys.path` and changes the working directory there. That is what allows later import statements like `src.datasets.meme_dataset` to work.

This pattern is common in research repositories. It is not elegant in a packaged Python-library sense, but it is pragmatic for notebook-based workflows.

3.5 Cell Group 4: Device Check

This cell checks the PyTorch version, CUDA availability, GPU count, and GPU memory. The purpose is not only convenience. It determines what later notebooks can realistically do.

If training later uses CLIP embeddings or multimodal fusion, GPU acceleration can change runtime from hours to minutes. The notebook also defines:

$$\text{DEVICE} = \begin{cases} \texttt{cuda} & \text{if a CUDA GPU exists} \\ \texttt{cpu} & \text{otherwise} \end{cases}$$

This is standard PyTorch practice.

3.6 Cell Group 5: Package Verification

The setup notebook checks imports for a list of modules and attempts `pip install` for anything missing. The package list includes core ML libraries plus interactive notebook packages and demo-related dependencies.

This is helpful for convenience, but it also reveals the real dependency footprint of the project. From a learning perspective, this cell teaches two things:

1. notebooks are executable environments, not just documents,
2. reproducibility depends on package management as much as on code correctness.

3.7 Cell Group 6: Dataset File Verification

The setup notebook validates the presence of train, dev, and test JSONL files and checks whether the image directory exists. It also counts how many non-empty lines are present in each split file.

For this workspace, direct dataset extraction confirms the expected split sizes:

Split	Samples	Notes
Train	8,500	labeled
Dev	500	labeled
Test	1,000	evaluation split present in workspace

The notebook also checks that the image directory is populated. In the local data snapshot used for this report, all 10,000 image references were present and no corrupted images were found.

3.8 Cell Group 7: Load One Sample

One of the most pedagogically useful steps in notebook 00 is the quick sample visualization. It loads the first non-empty record in the train file, resolves the referenced image path, prints the label and text, and displays the image.

This is a classic sanity check in multimodal projects because it validates all of the following at once:

- JSON parsing works,
- image path resolution works,
- the label field exists,
- the text field exists,
- the image can actually be rendered.

This is much better than only checking file existence. A file can exist and still be semantically unusable.

3.9 Cell Group 8: Source Module Import Check

The notebook tries to import classes from `src.datasets.meme_dataset` and `src.models.clip_encoder`. That tells the reader the broader repository is intended to include custom dataset and encoder abstractions, not only ad hoc notebook code.

If this cell fails, the problem is usually one of three things:

- the repo root was not detected correctly,
- the `src/` folder is missing,
- or the codebase is only partially present.

3.10 Cell Group 9: Config and Checkpoint Verification

The last setup cells look for configuration files and saved `.pt` checkpoints. These checks are optional, but they are helpful because they answer whether the project is in a fresh state or a partially trained state.

3.11 What Notebook 00 Teaches

The setup notebook is not teaching modeling directly. It is teaching **runtime hygiene**. A student reading it carefully should come away with these lessons:

- do not trust a notebook until paths, packages, and files are validated,
- make notebooks portable across environments,
- check one real sample before running large experiments,
- treat import failures as environment problems first, not as model problems.

Chapter 4

Notebook 01: Problem Scope and Design Decisions

Notebook 01 is mostly conceptual. That is good design. Before building a model, the project explicitly defines what it is trying to solve, what it is *not* trying to solve, what success means, and why a particular architecture family was chosen.

4.1 Task Definition

The notebook defines the problem as binary multimodal classification: hateful vs non-hateful. It also explains why the project does not immediately attempt a multi-class taxonomy such as sarcasm, offensiveness, or implicit hate.

The reason is sound: the dataset does not natively provide that richer annotation scheme. So the project chooses a tractable supervised objective first.

This is good scientific scoping. A model can only learn the distinctions the labels support.

4.2 Scope Boundary

The notebook says the project focuses on:

- overt hate speech in memes,
- hate that requires both image and text to identify,
- not general offensiveness or unrelated sarcasm.

This boundary matters. Without it, readers may over-claim what the system can do. In responsible AI work, that is dangerous. A model trained for one moderation task should not be casually marketed as solving all abuse-detection tasks.

4.3 Dataset Description

Notebook 01 summarizes the Meta AI Hateful Memes Challenge dataset and emphasizes a crucial point: it contains **confounder-style construction**. That means the same or similar visual contexts can pair with different captions and different labels.

The conceptual lesson is that the dataset is intentionally designed to punish shallow shortcuts. If a model learns only stereotypical image features or only toxic keywords, it should fail on hard examples.

4.4 Success Metrics

The notebook lists AUROC as the primary metric and also tracks Macro F1, hateful-class F1, and accuracy.

This is the correct ranking of priorities for moderation-style systems. AUROC matters because threshold selection may change by deployment context. If an application wants aggressive filtering, it may operate at a different threshold than a system that wants fewer false positives.

In notation, if a model produces a score $s(x)$ and a threshold τ, then:

$$\hat{y} = \mathbb{1}[s(x) \geq \tau]$$

Accuracy depends on a specific choice of τ. AUROC does not.

4.5 Architecture Decisions

This notebook makes one of the project's most important design commitments: use CLIP-based image and text encoders plus cross-attention fusion.

The reasoning is coherent:

- CLIP already aligns images and text in a shared representation space.
- A separate text encoder is unnecessary if CLIP's text encoder is strong enough.
- Cross-attention is preferred over naive concatenation because it can model token-level or feature-level interaction between modalities.
- Frozen encoders reduce compute cost and lower the risk of catastrophic forgetting under limited GPU budget.

The notebook also states a baseline ladder. This is not just engineering convenience. It is a scientific control structure. The project wants to show that each increase in multimodal sophistication adds value over simpler baselines.

4.6 Imbalance Handling

The notebook proposes weighted random sampling and focal loss. These choices respond to class imbalance and hard-example concentration.

If p_t is the predicted probability assigned to the true class, focal loss is often written as:

$$\mathcal{L}_{focal} = -\alpha(1 - p_t)^{\gamma} \log(p_t)$$

The term $(1-p_t)^{\gamma}$ suppresses easy examples and emphasizes hard ones. This is especially useful when a dataset contains many easy negatives but a smaller number of subtle or ambiguous positives.

4.7 Responsible AI Layer

Notebook 01 is also where the project commits to explainability, counterfactuals, fairness-style subgroup evaluation, calibration, and human-in-the-loop routing.

This is significant because it moves the project beyond pure benchmark chasing. The goal is not merely to get one score. The goal is to build a moderation-oriented system that can be inspected, challenged, and escalated to a human when appropriate.

4.8 Deliverables Framing

The notebook lists concrete deliverables such as a research prototype, explainability components, counterfactual analysis, a web demo, a backend API, an audit log, and an error analysis report.

That turns the project from a single notebook experiment into an end-to-end system design. For a student, this is a useful reminder that real ML work usually spans data validation, modeling, interpretability, deployment surfaces, and auditability.

4.9 Final Sanity Check Cell

Notebook 01 ends with a compact code cell that re-detects the dataset path and prints one sample schema. This is a deliberate pattern throughout the project: each notebook is able to run independently rather than depending on hidden state from a previous notebook.

That design choice improves robustness and makes each notebook easier to grade, debug, and re-run in isolation.

Chapter 5

Notebook 02: Data Inspection and Verification

Notebook 02 moves from conceptual planning to empirical validation of the raw data. It asks: what exactly is in the dataset we are about to model?

5.1 Why Data Inspection Comes Before EDA

Data inspection is narrower than EDA. Inspection checks integrity and structure. EDA asks broader analytical questions. That separation is healthy. Before interpreting patterns, we should first establish that the data loads correctly and is internally consistent.

5.2 Loading the Splits

The notebook repeats the robust dataset detection logic, then loads train, dev, and test JSONL files into DataFrames. This repetition may seem redundant, but it supports notebook independence.

From the workspace snapshot used for this report, the dataset sizes are:

Split	Count	Interpretation
Train	8,500	model fitting split
Dev	500	tuning and validation split
Test	1,000	held-out evaluation data present in workspace

5.3 Image Path Resolution

The helper function in notebook 02 is more sophisticated than a simple `os.path.join`. It tries multiple candidate layouts because image paths may be stored with or without `img/` prefixes and may differ between local and Kaggle setups.

That matters because dataset portability problems often emerge from tiny path-format inconsistencies. The helper explicitly encodes those variants rather than assuming one canonical layout.

5.4 Class Balance Check

The notebook prints per-class counts for the train and dev sets. In this dataset snapshot:

Split	Non-hateful	Hateful	Comment
Train	5,450	3,050	moderate imbalance
Dev	250	250	perfectly balanced

The train split therefore has a non-hateful to hateful ratio of approximately:

$$\frac{5450}{3050} \approx 1.79 : 1$$

That imbalance is not extreme, but it is large enough that loss weighting, focal loss, or balanced sampling are reasonable design responses.

5.5 Image Existence and Corruption Check

The notebook iterates through all samples and verifies that the referenced image exists and is readable. This is vital because image-text tasks depend on both modalities being present.

For the local dataset used here:

- missing images: 0
- corrupted images: 0
- total samples checked: 10,000

This is a strong sign that the project data bundle is internally consistent at the file level.

5.6 Duplicate and Leakage Checks

Notebook 02 hashes image files and text strings to estimate duplicates and train-dev overlap. This is one of the most important quality-control steps because leakage can make a model appear stronger than it really is.

In the train split, a direct hash-based pass found 1,428 duplicated text strings. That does not automatically mean label leakage, but it does mean the text distribution contains repetition. Such repetition can encourage lexical shortcut learning.

The notebook also contains logic to check overlapping image hashes between train and dev. This is precisely the kind of integrity check that should happen before benchmarking any model.

5.7 Missing or Empty Text Fields

The notebook checks for null and empty text fields in every split. In the local snapshot used for this report, all three splits had zero null text entries and zero empty text entries.

That result supports the notebook 04 decision to treat the dataset OCR text as generally reliable.

5.8 Random Sample Inspection

The notebook displays random hateful and non-hateful examples. This is not merely for presentation value. Human inspection helps answer questions that simple statistics cannot:

- does the label distribution feel plausible,
- do some samples look ambiguous,
- are there obvious annotation artifacts,
- does the hateful class visibly require multimodal reasoning.

If a student skips this step, they often miss the semantic difficulty of the problem.

5.9 Image Resolution Analysis

Notebook 02 samples image sizes and plots width and height distributions. On a sample of 200 training images extracted directly from the workspace data, the observed summary was:

Quantity	Min	Max	Mean
Width	247	825	608.59
Height	309	804	527.03

This tells the reader that image sizes are moderately variable but not wildly unconstrained. That matters when later deciding on resize policies, CLIP input resolution, and augmentation settings.

5.10 Modality-Need Analysis

The notebook computes text length and explores whether very short texts may be image-reliant and longer texts may be text-dominant. This is a simple heuristic, but it is conceptually useful.

From the local train split:

- mean word count overall: 11.74

- mean word count for non-hateful: 11.15
- mean word count for hateful: 12.79
- median word count: 10
- 95th percentile word count: 24
- mean character count overall: 62.08

These values support a practical conclusion: meme text is usually short enough that OCR quality and token-level signal matter a lot.

5.11 What Notebook 02 Teaches

This notebook teaches the difference between **having data** and **trusting data**. A good machine learning workflow does not jump from files on disk straight to model training. It verifies class balance, file consistency, possible duplicates, and the general shape of both modalities.

Chapter 6

Notebook 03: Exploratory Data Analysis

Notebook 03 is the broad analytical notebook. Its goal is not just to verify the data exists, but to learn how the data behaves. It explores label balance, text lengths, lexical patterns, duplication, surface forms, demographic mentions, and confounder-like structure.

6.1 Why EDA Matters Here

In multimodal hate detection, poor EDA can lead to poor modeling decisions. For example:

- If hateful texts are much longer, the model might exploit length as a shortcut.
- If certain identity words dominate the hateful class, the model may over-associate mentions of protected groups with hate.
- If exact text duplicates exist, train performance can be inflated.
- If image reuse exists, the image encoder may memorize content patterns rather than reasoning compositionally.

Notebook 03 tries to surface these possibilities.

6.2 Class Distribution

The notebook begins with bar and pie plots for train-label distribution. This is partly redundant with notebook 02, but useful in EDA because it visually frames later analyses.

The train split imbalance ratio is about $1.79 : 1$ in favor of the non-hateful class. That means any naive classifier that leans too hard toward the majority class can still look deceptively reasonable on accuracy.

6.3 Text Length Distributions

The notebook computes word count and character count for each sample and compares the two classes. These are simple features, but they often reveal shortcut risks.

A key insight from the current dataset snapshot is:

$$\text{mean word count}_{hate} > \text{mean word count}_{nonhate}$$

Numerically:

$$12.79 > 11.15$$

That gap is not huge, but it is real enough that a careless model might partially exploit it. The notebook later strengthens this analysis by using the Mann-Whitney U test to check whether such differences are statistically significant.

6.4 Detailed Text Statistics

Notebook 03 goes beyond simple means. It computes medians, standard deviations, quantiles, and average word length. This is useful because hateful-meme text is not normally distributed, and means alone can hide skewness.

For example, the 95th percentile word count of 24 suggests most memes are textually brief. This supports later design choices such as transformer-based text encoding with short token sequences.

6.5 Violin Plots and Statistical Testing

The notebook compares distributions with violin plots and runs a Mann-Whitney U test. This is a nonparametric test, which is appropriate when normality assumptions may not hold.

The core question is whether the observed class differences in features such as word count or character count are likely due to random fluctuation or reflect a real distributional shift.

That shift matters because if basic text-shape features already separate the classes somewhat, then a model may learn shallow heuristics unless the evaluation and architecture force it to use richer cross-modal evidence.

6.6 Vocabulary Richness

Notebook 03 computes counts such as total tokens, unique words, type-token ratio, and hapax legomena. These measures help estimate how repetitive or diverse the language is.

If one class has a much lower type-token ratio, it may indicate more repetitive formulaic phras-

ing. If hateful memes repeatedly use a narrow vocabulary, a text-only classifier may perform surprisingly well on obvious cases while still failing on nuanced multimodal cases.

6.7 Zipf-Law Visualization

The notebook plots ranked word frequencies on log-log axes. This is a standard NLP sanity check because natural language typically follows an approximate Zipfian pattern: a few words occur often, many words occur rarely.

This does not solve a problem directly, but it helps verify that the token distribution looks language-like rather than corrupted or pathologically preprocessed.

6.8 Bigram and Trigram Analysis

The n-gram section extracts common bigrams and trigrams after removing basic stopwords. This is a useful middle ground between individual word counts and full semantic modeling.

Single words can be misleading. Phrases are often more revealing. For moderation tasks, harmful meaning is frequently phrase-based rather than token-based.

For example, identity words alone do not imply hate. Certain multi-word combinations do.

6.9 Surface Characteristics

The notebook measures uppercase ratio, punctuation count, question marks, exclamation marks, digit count, and all-caps frequency. These are weak features individually, but together they help profile the rhetorical style of the corpus.

This kind of analysis is useful for two reasons:

1. it can reveal easy spurious cues,
2. it can inform preprocessing choices such as whether to preserve case and punctuation.

If hateful examples rely disproportionately on all-caps shouting or repeated punctuation, a text model may exploit that without understanding semantics.

6.10 Text Duplication and Overlap

One of the strongest parts of notebook 03 is its explicit duplication analysis. It counts exact duplicate texts, cross-label duplicates, train-dev text overlap, and image reuse.

This is not just housekeeping. It goes directly to the credibility of any later reported model score.

When the same text appears multiple times, two kinds of risk emerge:

- overfitting to repeated textual patterns,
- accidental leakage if similar items span train and validation splits.

The notebook also warns about cross-label duplicates, which can indicate label noise or genuinely context-dependent multimodal cases.

6.11 Word Frequencies and Word Clouds

The notebook includes top-word charts and word clouds for each class. These visualizations are easy to read but easy to misuse. A careful reading should keep two constraints in mind:

- frequent words are not necessarily causal features,
- identity terms can appear in both benign and hateful contexts.

So the right use of these plots is diagnostic, not normative. They show what the corpus contains; they do not justify simplistic decision rules.

6.12 Demographic Mention Analysis

Notebook 03 defines hand-crafted keyword lists for groups such as race, gender, religion, disability, and sexuality, then measures mention rates by label. This is a rough but useful fairness-oriented probe.

The idea is not to declare bias from one chart. The idea is to discover whether certain protected-group terms are heavily entangled with label assignment, which later informs auditing and calibration.

This is especially important in moderation systems because false positives involving identity mentions are ethically and practically costly.

6.13 Confounder Check

The notebook checks whether the same image appears with both labels, treating such cases as confounder evidence. Conceptually, that is a smart test: if the same image is reused with different texts and different labels, the dataset directly demonstrates that image-only reasoning is insufficient.

In the direct workspace-level extraction used while preparing this report, no such same-image two-label cases were found within the train split. That does not invalidate the notebook's intent. It simply means the local snapshot or the exact split being checked may not surface those cases in the way the markdown discussion anticipates.

This is a good lesson in itself: EDA code often encodes a hypothesis that should be checked, not assumed.

6.14 What Notebook 03 Teaches

Notebook 03 teaches a mature ML habit: before choosing a model, understand the signal landscape and the shortcut landscape. It helps the reader see:

- how separable the classes may already be from shallow lexical features,
- where duplication or leakage might live,
- why fairness and confounding must be part of analysis,
- why multimodal reasoning remains necessary even when text statistics are informative.

Chapter 7

Notebook 04: OCR Pipeline

Notebook 04 answers a pragmatic question: should the project trust the dataset's provided text field, replace it with OCR, or combine both?

7.1 The Core Tension

The Meta Hateful Memes dataset already provides a text field described as human-cleaned OCR. That is usually better than running OCR blindly yourself. But the notebook still audits OCR quality because some examples may be too short, suspicious, or incomplete.

The notebook therefore does not start from the assumption that OCR is always necessary. It starts from the assumption that OCR quality should be verified empirically.

7.2 Step 1: Audit Existing Text Quality

The notebook computes simple diagnostics:

- whether text exists,
- whether text is very short,
- whether it contains numbers,
- whether it contains unusual characters,
- average word count,
- average character count.

For the local train split used in this report, the extracted audit values were:

Statistic	Value
Samples with non-empty text	8,500
Very short text ($\leq$ 2 words)	153
Contains numbers	0
Contains special characters	1,010
Average word count	11.74
Average character count	62.08

These numbers strongly support the claim that the provided text field is generally populated and useful.

7.3 Step 2: Inspect Suspiciously Short Entries

The notebook samples memes with one word or less in the OCR text. This is smart because averages can hide edge cases. If the text field fails, it is more likely to fail on these short or empty cases than on normal cases.

7.4 Step 3: Install and Initialize EasyOCR

Notebook 04 conditionally installs `easyocr`, checks whether CUDA is available, and initializes an English OCR reader.

This is the first notebook in the sequence that introduces a heavier external dependency at runtime. It also reveals an engineering tradeoff:

- OCR can improve coverage,
- but OCR is computationally expensive,
- and the provided dataset text may already be better than raw OCR output.

7.5 Step 4: Compare Dataset Text With EasyOCR on a Sample

The notebook runs EasyOCR on 20 sampled images and stores both the dataset text and OCR output. It cleans OCR text by collapsing whitespace and then computes a simple overlap score.

The overlap score is heuristic rather than rigorous, but that is acceptable for the notebook's stated goal. It is not trying to publish a new OCR benchmark. It is trying to decide whether OCR replacement is worth the cost.

7.6 Step 5: OCR Quality Heuristic

The overlap function is based on word-set intersection. If the dataset text and EasyOCR text share many words, then the existing text field is likely already adequate.

This approach is deliberately simple. It does not account for semantic equivalence, word order, or partial recognitions. But as a quick diagnostic, it is enough to support an engineering decision.

7.7 Step 6: Final Decision Logic

The notebook's conclusion is conservative and sensible:

- use the dataset `text` field as the primary source,
- use EasyOCR only as a fallback when the dataset text is too short or missing,
- otherwise avoid a full re-OCR pass.

The helper function `get_best_text` encodes exactly that policy. It prioritizes the existing dataset text if it has at least a minimum number of words, falls back to EasyOCR if needed, and otherwise returns whichever source is longer.

This is a practical decision rule for downstream preprocessing because it balances quality, compute cost, and reproducibility.

7.8 Step 7: Optional Full OCR Pass

The notebook includes a disabled branch for running EasyOCR over all splits and writing an `enhanced_text` field to disk. This is useful for experimentation but intentionally off by default because it can take 20 to 30 minutes on a T4 GPU.

That is a good example of notebook design. Expensive steps are available, but not silently triggered.

7.9 Step 8: Visual Overlay Output

The final notebook cell displays sampled images with OCR text overlays and label coloring. This is pedagogically valuable because it reconnects the OCR audit to actual memes rather than leaving it as a purely numeric exercise.

7.10 What Notebook 04 Teaches

This notebook teaches a broader engineering lesson: do not replace curated data with automated extraction unless you have evidence that the replacement is superior. In this project, the dataset's human-cleaned text is treated as authoritative by default, and OCR is a fallback mechanism rather than a default preprocessing stage.

Chapter 8

Cross-Notebook Patterns and Design Lessons

These notebooks share a few design patterns worth making explicit.

8.1 Each Notebook Is Self-Sufficient

Instead of relying on hidden state from previous notebooks, the code repeatedly re-detects dataset paths and reloads splits. That increases repetition, but it improves reliability. A notebook should ideally run correctly when opened on its own.

8.2 Environment Robustness Is Treated as Part of the Project

The project does not assume a perfect local machine. It supports Kaggle and local execution, CPU and GPU fallbacks, and multiple possible folder layouts. That makes the notebooks more robust and more transferable.

8.3 Analysis Precedes Modeling

This sequence is structurally sound:

1. verify the runtime,
2. define the task,
3. inspect data integrity,
4. study data behavior,
5. audit OCR quality,
6. only then move into preprocessing and modeling.

That order reduces the chance of building a model on top of misunderstood or degraded inputs.

8.4 The Project Is Already Framed as More Than a Benchmark

From notebook 01 onward, the repository is framed as an end-to-end moderation-aware system, not just a classifier. Explainability, counterfactuals, and human review are part of the planned pipeline, which is a mature framing for a socially sensitive task.

Chapter 9

Concise Notebook-by-Notebook Summary

Notebook	Main question	What the reader should learn
00_setup	Is the environment trustworthy?	How to validate paths, packages, devices, data files, imports, and checkpoints before doing analysis.
01_problem-scope	What exactly are we building, and why?	How to define the task, pick metrics, justify CLIP plus fusion, and set boundaries for responsible use.
02_data-inspection	What data do we actually have?	How to verify split sizes, class balance, image integrity, duplication, and raw sample quality.
03_eda	What patterns and risks exist in the corpus?	How to study class imbalance, text distributions, duplication, lexical signals, subgroup mentions, and shortcut risks.
04_ocr_pipeline	Should we trust dataset text or rerun OCR?	How to audit text quality, sample EasyOCR, compare outputs, and adopt a fallback-only OCR strategy.

Chapter 10

Final Interpretation

If someone reads these notebooks carefully from the beginning, the main intellectual story is this:

1. The project first establishes that the environment and dataset are valid.
2. It then defines hateful meme detection as a multimodal binary classification problem with responsible-AI constraints.
3. It verifies that the dataset is structurally clean: the split files load, image files exist, and text fields are populated.
4. It studies the corpus deeply enough to identify imbalance, repetition, lexical skew, and subgroup-sensitive patterns.
5. It confirms that the provided text field is good enough to use as the main textual signal, with OCR reserved for fallback cases.

That progression is disciplined. It prevents premature modeling and gives later training notebooks a strong analytical foundation.

Chapter A

Key Numbers Used in This Report

The following values were extracted directly from the local dataset snapshot while preparing this guide:

- Split sizes: train 8,500; dev 500; test 1,000.
- Train balance: 5,450 non-hateful, 3,050 hateful.
- Dev balance: 250 non-hateful, 250 hateful.
- Missing images: 0.
- Corrupted images: 0.
- Duplicate train text strings by hash: 1,428.
- Mean word count: 11.74 overall, 11.15 non-hateful, 12.79 hateful.
- Median word count: 10.
- 95th percentile word count: 24.
- Mean character count: 62.08.
- OCR audit: all 8,500 training samples had non-empty text; 153 had two words or fewer.

Chapter B

Suggested Reading Order for a Beginner

If you are new to this domain, the most effective reading order is:

1. Read notebook 01 first to understand the problem definition.
2. Read notebook 00 second so you understand how the runtime is validated.
3. Read notebook 02 to learn the raw dataset structure.
4. Read notebook 03 slowly, because it contains the analytical intuition behind later modeling choices.
5. Read notebook 04 last to understand why the project trusts the dataset text more than fresh OCR.

That order is slightly different from execution order, but often better for conceptual understanding.